Optimize GitLab CI/CD configuration files

Tier: Free, Premium, Ultimate

Offering: GitLab.com, GitLab Self-Managed, GitLab Dedicated

You can reduce complexity and duplicated configuration in your GitLab CI/CD configuration files by using:

- YAML-specific features like anchors (`&`), aliases (`*`), and map merging (`<<`). Read more about the various YAML features.
- The `extends` keyword, which is more flexible and readable. You should use `extends` where possible.

To create multiple similar jobs, but with different variable values, use `parallel:matrix.

Anchors

YAML has a feature called 'anchors' that you can use to duplicate content across your document.

You can use anchors to duplicate or inherit properties. Use anchors with hidden jobs to provide templates for your jobs.

The `&` character marks the anchor name, and the `*` character is the alias that references the anchor. You must define the anchor higher in the YAML file than any aliases that reference it.

When there are duplicate keys, the latest included key wins and overrides the other keys.

In certain cases (see YAML anchors for scripts), you can use YAML anchors to build arrays with multiple components defined elsewhere. For example:

```yaml
.default_scripts: &default_scripts
  - ./default-script1.sh
  - ./default-script2.sh

job1:
  script:
    - *default_scripts
    - ./job-script.sh
```

You can't use YAML anchors across multiple files when using the `include` keyword. Anchors are only valid in the file they were defined in. To reuse configuration from different YAML files, use `!reference` tags or the `extends` keyword.

The following example uses anchors and map merging. It creates two jobs, `test1` and `test2`, that inherit the `.job_template` configuration, each with their own custom `script` defined:

```yaml
```

```
  services:
    - postgres
    - redis

test1:
  <<: *job_configuration           # Add the contents of the 'job_configuration' alias
  script:
    - test1 project

test2:
  <<: *job_configuration           # Add the contents of the 'job_configuration' alias
  script:
    - test2 project
```

`&` sets up the name of the anchor (`job_configuration`), `<<` means "merge the given hash into the current one," and `*` includes the named anchor (`job_configuration` again). The expanded version of this example is:

yaml

```yaml
.job_template:
  image: ruby:2.6
  services:
    - postgres
    - redis

test1:
  image: ruby:2.6
  services:
    - postgres
    - redis
  script:
    - test1 project

test2:
  image: ruby:2.6
  services:
    - postgres
    - redis
  script:
    - test2 project
```

You can use anchors to define two sets of services. For example, `test:postgres` and `test:mysql` share the `script` defined in `.job_template`, but use different `services`, defined in `.postgres_services` and `.mysql_services`:

yaml

```
  tags:
    - dev

.postgres_services:
  services: &postgres_configuration
    - postgres
    - ruby

.mysql_services:
  services: &mysql_configuration
    - mysql
    - ruby

test:postgres:
  <<: *job_configuration
  services: *postgres_configuration
  tags:
    - postgres

test:mysql:
  <<: *job_configuration
  services: *mysql_configuration
```

The expanded version is:

```yaml
.job_template:
  script:
    - test project
  tags:
    - dev

.postgres_services:
  services:
    - postgres
    - ruby

.mysql_services:
  services:
    - mysql
    - ruby

test:postgres:
  script:
    - test project
  services:
    - postgres
    - ruby
  tags:
    - postgres

test:mysql:
  script:
    - test project
  services:
    - mysql
    - ruby
  tags:
    - dev
```

You can see that the hidden jobs are conveniently used as templates, and `tags: [postgres]` overwrites `tags: [dev]`.

History >

You can use YAML anchors with script, `before_script`, and `after_script` to use predefined commands in multiple jobs:

yaml

```yaml
.some-script-before: &some-script-before
  - echo "Execute this script first"

.some-script: &some-script
  - echo "Execute this script second"
  - echo "Execute this script too"

.some-script-after: &some-script-after
  - echo "Execute this script last"

job1:
  before_script:
    - *some-script-before
  script:
    - *some-script
    - echo "Execute something, for this job only"
  after_script:
    - *some-script-after

job2:
  script:
    - *some-script-before
    - *some-script
    - echo "Execute something else, for this job only"
    - *some-script-after
```

Use `extends` to reuse configuration sections

You can use the `extends` keyword to reuse configuration in multiple jobs. It is similar to YAML anchors, but simpler and you can use `extends` with `includes`.

`extends` supports multi-level inheritance. You should avoid using more than three levels, due to the additional complexity, but you can use as many as eleven. The following example has two levels of inheritance:

yaml

```yaml
.rspec:
  extends: .tests
  script: rake rspec

rspec 1:
  variables:
    RSPEC_SUITE: '1'
  extends: .rspec

rspec 2:
  variables:
    RSPEC_SUITE: '2'
  extends: .rspec

spinach:
  extends: .tests
  script: rake spinach
```

Exclude a key from `extends`

To exclude a key from the extended content, you must assign it to `null`, for example:

```yaml
.base:
  script: test
  variables:
    VAR1: base var 1

test1:
  extends: .base
  variables:
    VAR1: test1 var 1
    VAR2: test2 var 2

test2:
  extends: .base
  variables:
    VAR2: test2 var 2

test3:
  extends: .base
  variables: {}

test4:
  extends: .base
  variables: null
```

Merged configuration:

```yaml
```

```yaml
    VAR1: test1 var 1
    VAR2: test2 var 2

test2:
  script: test
  variables:
    VAR1: base var 1
    VAR2: test2 var 2

test3:
  script: test
  variables:
    VAR1: base var 1

test4:
  script: test
  variables: null
```

Use `extends` and `include` together

To reuse configuration from different configuration files, combine `extends` and `include`.

In the following example, a `script` is defined in the `included.yml` file. Then, in the `.gitlab-ci.yml` file, `extends` refers to the contents of the `script`:

- `included.yml`:

```yaml
.template:
  script:
    - echo Hello!
```

- `.gitlab-ci.yml`:

```yaml
include: included.yml

useTemplate:
  image: alpine
  extends: .template
```

Merge details

You can use `extends` to merge hashes but not arrays. When there are duplicate keys, GitLab performs a reverse deep merge based on the keys. Keys from the last member always override anything defined on other levels. For example:

```yaml
```

```yaml
    IMPORTANT_VAR: "the details"
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
    - if: $CI_COMMIT_BRANCH == "stable"
  tags:
    - production
  script:
    - echo "Hello world!"

.in-docker:
  variables:
    URL: "http://docker-url.internal"
  tags:
    - docker
  image: alpine

rspec:
  variables:
    GITLAB: "is-awesome"
  extends:
    - .only-important
    - .in-docker
  script:
    - rake rspec
```

The result is this `rspec` job:

```yaml
rspec:
  variables:
    URL: "http://docker-url.internal"
    IMPORTANT_VAR: "the details"
    GITLAB: "is-awesome"
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
    - if: $CI_COMMIT_BRANCH == "stable"
  tags:
    - docker
  image: alpine
  script:
    - rake rspec
```

In this example:

- The `variables` sections merge, but `URL: "http://docker-url.internal"` overwrites `URL: "http://my-url.internal"`.
- `tags: ['docker']` overwrites `tags: ['production']`.
- `script` does not merge, but `script: ['rake rspec']` overwrites `script: ['echo "Hello world!"']`. You can use YAML anchors to merge arrays.

`!reference` tags

Use the `!reference` custom YAML tag to select keyword configuration from other job sections and reuse it in the current section. Unlike YAML anchors, you can use `!reference` tags to reuse configuration from included configuration files as well.

If you are using `!reference` tags to override configuration from included files, consider using CI/CD inputs instead. You cannot use CI/CD inputs in `!reference` tags, because `!reference` tags are evaluated before input interpolation.

In the following example, a `script` and an `after_script` from two different locations are reused in the `test` job:

- `configs.yml`:

```
  script:
    - echo creating environment
```

- `.gitlab-ci.yml`:

```yaml
include:
  - local: configs.yml

.teardown:
  after_script:
    - echo deleting environment

test:
  script:
    - !reference [.setup, script]
    - echo running my own command
  after_script:
    - !reference [.teardown, after_script]
```

In the following example, `test-vars-1` reuses all the variables in `.vars`, while `test-vars-2` selects a specific variable and reuses it as a new `MY_VAR` variable.

```yaml
.vars:
  variables:
    URL: "http://my-url.internal"
    IMPORTANT_VAR: "the details"

test-vars-1:
  variables: !reference [.vars, variables]
  script:
    - printenv

test-vars-2:
  variables:
    MY_VAR: !reference [.vars, variables, IMPORTANT_VAR]
  script:
    - printenv
```

You can use multiple `!reference` tags to build up an array with `rules`, `script`, or stages. For example:

```yaml
.rules_prod:
  - if: $CI_PIPELINE_SOURCE == "merge_request_event"
  - if: $CI_PIPELINE_SOURCE == "schedule"

.rules_staging:
  - if: $CI_COMMIT_BRANCH =~ /^wip-.*/
  - if: $CI_PIPELINE_SOURCE == "push"

deploy_job:
  script: echo test
  rules:
    - !reference [.rules_prod]
    - !reference [.rules_staging]
```

```yaml
.snippets:
  one:
    - echo "ONE!"
  two:
    - !reference [.snippets, one]
    - echo "TWO!"
  three:
    - !reference [.snippets, two]
    - echo "THREE!"

nested-references:
  script:
    - !reference [.snippets, three]
```

In this example, the `nested-references` job runs all three `echo` commands.

Configure your IDE to support `!reference` tags

The pipeline editor supports `!reference` tags. However, the schema rules for custom YAML tags like `!reference` might be treated as invalid by your editor by default. You can configure some editors to accept `!reference` tags. For example:

- In VS Code, you can set `vscode-yaml` to parse `customTags` in your `settings.json` file:

```json
"yaml.customTags": [
  "!reference sequence"
]
```

- In Sublime Text, if you are using the `LSP-yaml` package, you can set `customTags` in your `LSP-yaml` user settings:

```json
{
  "settings": {
    "yaml.customTags": ["!reference sequence"]
  }
}
```